

Cost-Sensitive Learning for Myocardial Infarction Mortality Prediction: A Static and Dynamic Ensemble Framework

Leonardo Saccotelli

Università degli Studi di Bari Aldo Moro

The Core Challenge in Clinical Prognosis

Predicting Myocardial Infarction Outcome

- Early risk stratification of Myocardial Infarction (MI) at hospital admission is crucial for timely ICU resource allocation and patient survival.
- Clinical data inherently suffers from severe class imbalance.
- Standard ML algorithms naturally ignore the minority to maximize global accuracy.

The Problem of Asymmetric Costs

The real-world clinical cost of prediction errors is not equal:

- **False Positive (FP):** Predicting a lethal outcome for a survivor → A false alarm leading to over-triage (manageable resource cost).
- **False Negative (FN):** Predicting survival for a patient who ultimately dies → A missed intervention with **catastrophic consequences**.

Research Objectives & Framework

Primary Objective

To identify the most robust predictive pipeline for admission-time MI mortality by explicitly accounting for both minority-class rarity and asymmetric misclassification costs.

A Unified Experimental Comparison

We systematically evaluate models across two main dimensions:

1. **Model Architectures:** Linear baseline, *Static Ensembles*, *Dynamic Ensemble Selection* (DES) methods.
2. **Cost-Sensitive Interventions:**
 - **Learning-Level:** Data oversampling (SMOTE), class-frequency weighting, cost-matrix-driven weighting.
 - **Decision-Level:** Standard 0.5 threshold, *Minimum Expected Cost* (MEC) decision rule.

Dataset Description and Construction

Myocardial Infarction Complications Dataset [Golovenkin et al., 2020]

- **Source:** UCI Machine Learning Repository.
- **Scale:** 1700 records, 111 input features (demographics, ECG, medical history, labs).
- **Targets:** 12 variables (11 post-MI complications, 1 lethal outcome - LET_IS).
- **Temporal Structure:** multiple ICU time points (admission, 24h, 48h, 72h).

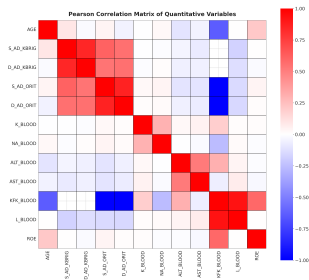
Time-Slice & Target Formulation

- **Admission-Time Focus:** Retained only admission time for early prediction.
- **Lethal Outcome Focus:** Retained only lethal outcome - LET_IS.
- **Binary Target Creation:** Collapsed the causes of death into a single positive class:
 - $\text{LET_IS} = 0 \rightarrow 0$ (**ALIVE**)
 - $\text{LET_IS} \geq 1 \rightarrow 1$ (**DEAD**)

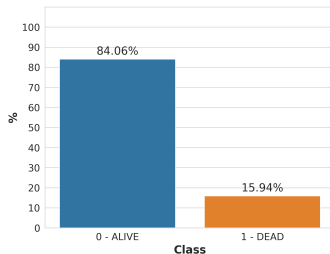
Exploratory Data Analysis (EDA)

Insights from the Data

- **Data Quality:** Average missing rate 7.64%, 7 features $>30\%$ missingness, 0 duplicates.
- **Numerical Features:** Some show normal distributions, while several lab parameters are right-skewed with outliers. Strongest correlations are localized to blood pressure features.
- **Categorical Features:** Several features are imbalanced, with frequencies concentrated in a few categories.
- **Target Variable (LET_IS):** Severely imbalanced, dominated by the **ALIVE (84.06%)** class.



Class Distribution for complication: LET_IS



Data Cleaning & Preprocessing (1/2)

Missing Data Handling

- **Filtering:** Records with $>20\%$ missing values, features with $>30\%$ missing values.
- **Imputation:** Median for numerical features, Mode for categorical features.

Type-Specific Feature Preprocessing

- **Numerical Feature:**
 - Heavily right-skewed distribution $\rightarrow \log1p + \text{StandardScaler}$.
 - Approximately normal distribution $\rightarrow \text{StandardScaler}$.
- **Categorical Feature:**
 - Nominal $\rightarrow \text{OneHotEncoder}$.
 - Binary $\rightarrow \text{LabelEncoder}$.
 - Ordinal $\rightarrow \text{OrdinalEncoder}$.

Data Cleaning & Preprocessing (2/2)

Custom Mapping (ZSN_A - Chronic Heart Failure)

- **Issue:** Partially ordered attribute with two intersecting severity lines ($0 < 1 < 2 < 4$ and $0 < 1 < 3 < 4$, where state 4 implies simultaneous states 2 and 3).
- **Solution:** Mapped into a 2D space to perfectly preserve severity structure:

$$0 \rightarrow [0, 0] \quad 1 \rightarrow [1, 1] \quad 2 \rightarrow [2, 1] \quad 3 \rightarrow [1, 2] \quad 4 \rightarrow [3, 3]$$

Feature Selection

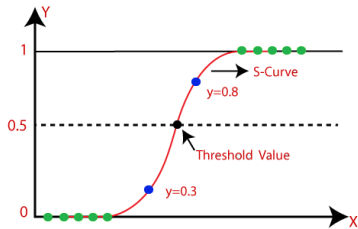
- Applied `SelectKBest` using Mutual Information as the scoring criterion.
- The number of retained features (k) was embedded in the learning pipeline as a strictly tunable hyperparameter.

Predictive Models: Logistic Regression

A Probabilistic Baseline

A foundational linear model that serves as our highly interpretable baseline.

$$P(y = 0|\mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^T \mathbf{x} - w_0)}$$



The logistic curve maps raw scores to predicted probabilities.

Model Setup & Hyperparameter Search Space

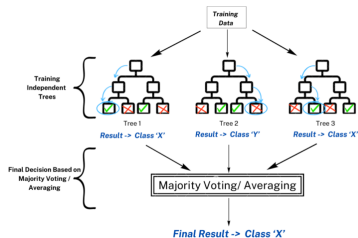
- solver: "liblinear", penalty: "l2", max_iter:1000
- $C \in [10^{-4}, 10^4)$, tol $\in [10^{-5}, 10^{-2})$, fit_intercept $\in \{\text{True}, \text{False}\}$

Predictive Models: Random Forest

Static Ensembles – Bagging Paradigm

An ensemble learning method that builds multiple decorrelated decision trees by learning them on a:

- randomly chosen subset of input variables,
- randomly chosen subset of data samples.



Model Setup & Hyperparameter Search Space

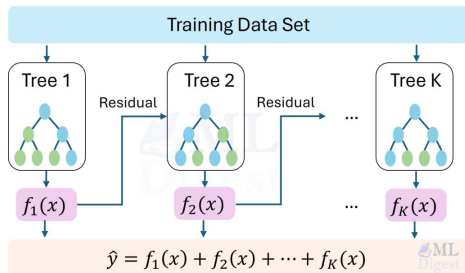
- criterion: "gini", bootstrap: "True"
- `n_estimators` $\in [100, 1000)$, `max_samples` $\in [0.5, 0.9)$, `max_depth` $\in [3, 20)$, `max_features` $\in \{\text{sqrt}, \text{log2}, \text{None}\}$, `min_samples_split` $\in [2, 10)$, `min_samples_leaf` $\in [1, 10)$, `max_leaf_nodes` $\in [2, 20)$, `ccp_alpha` $\in [0.0, 0.01)$, `min_impurity_decrease` $\in [0.0, 0.1)$

Predictive Models: XGBoost

Static Ensembles – Boosting Paradigm

XGBoost is an optimized gradient boosting algorithm that combines multiple decision trees as base learners, building them sequentially:

- each tree corrects errors of the previous one,
- sum the predictions of all the trees.



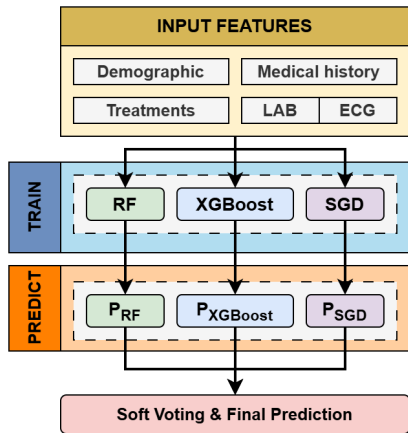
Model Setup & Hyperparameter Search Space

- objective: "binary:logistic", eval_metric: "logloss"
- `n_estimators` $\in [200, 800)$, `learning_rate` $\in [10^{-2}, 0.2)$, `max_depth` $\in [3, 10)$, `subsample` $\in [0.6, 1.0)$, `colsample_bytree` $\in [0.6, 1.0)$, `gamma` $\in [0.0, 5.0)$, `reg_alpha` $\in [10^{-4}, 10)$, `reg_lambda` $\in [10^{-4}, 10)$, `min_child_weight` $\in [1, 10)$

Predictive Models: Voting

Static Ensembles – Voting Paradigm

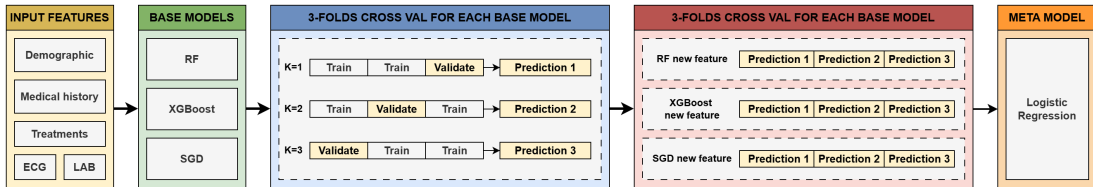
- **Heterogeneous Pool:** Random Forest, XGBoost, and SGD Classifier (loss="log_loss", alpha $\in [10^{-5}, 10)$, l1_ratio $\in [0, 1)$, penalty $\in \{l2, l1, elasticnet\}$).
- **Independent Tuning:** Each base learner is independently optimized.
- **Soft Voting :** Gathers the predicted probabilities from all base models and computes the simple mathematical average (uniform weights) to output a robust final decision.



Predictive Models: Stacking

Static Ensembles – Stacking Paradigm

- **Heterogeneous Pool:** Random Forest, XGBoost, and SGD Classifier.
- **Independent Tuning:** Each base learner is independently optimized.
- **Cross-Validated Inputs:** The base learners generate out-of-fold predictions using a $cv=3$ splitting strategy to prevent overfitting.
- **Meta-Estimator:** These intermediate predictions become the *new input features* for a higher-level algorithm (LogisticRegression).



Predictive Models: Dynamic Ensemble Selection (1/4)

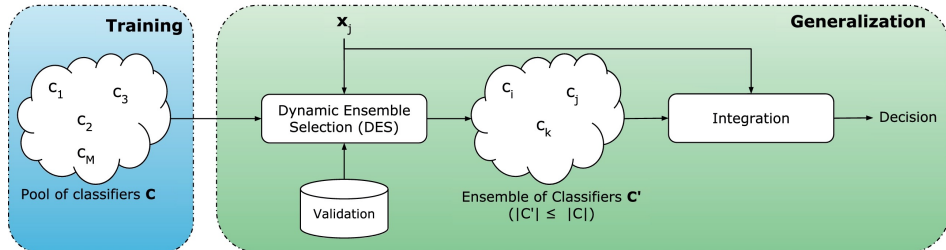
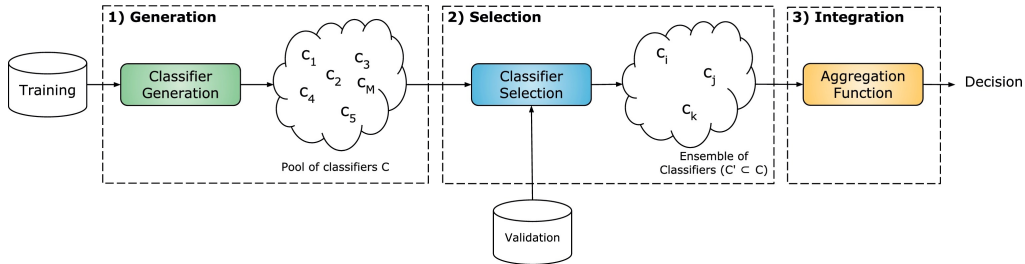
The Dynamic Paradigm [Cruz et al., 2018]

Unlike static models that rely on a fixed ensemble for all instances, DES selects the most competent subset of classifiers for *each specific query sample* based on its local context.

The 3-Phase Architecture

1. **Generation:** A diverse pool of classifiers is built.
2. **Selection (Competence Estimation):** For a new query, a local "region of competence" is defined using a dedicated dataset (**DSEL**). The system evaluates and selects the local experts.
3. **Integration:** The predictions of the dynamically selected subset are aggregated to form the final decision.

Predictive Models: Dynamic Ensemble Selection (2/4)



Predictive Models: Dynamic Ensemble Selection (3/4)

1. KNORA-E (Oracle-Based)

- Identifies local "experts" that *perfectly* classify all samples in the query's defined region of competence.
- Dynamically shrinks the neighborhood size (removes the furthest neighbor) if no perfect oracle is initially found.

2. DES-KL (Information Theory)

- Evaluates competence via a distance-weighted Gaussian potential function within the local k-NN neighborhood.
- Computes the Kullback–Leibler (KL) divergence between a base classifier's output profile and a Random Classifier. Models performing significantly better than random guessing are selected.

Predictive Models: Dynamic Ensemble Selection (4/4)

3. META-DES (Meta-Learning)

- Extracts advanced local meta-features (e.g., neighborhood consensus, model confidence) for each query.
- Utilizes a trained Multinomial Naive Bayes meta-classifier to explicitly predict if a base-learner is competent enough to be selected.

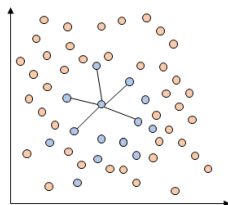
Implementation Specifics

- **Base Classifier Pool:** BaggingClassifier of DecisionTreeClassifier
 - `n_estimators` $\in [100, 1000)$, `max_samples` $\in [0.5, 0.9)$, `max_depth` $\in [3, 20)$, `min_samples_split` $\in [2, 10)$, `min_samples_leaf` $\in [1, 10)$, `ccp_alpha` $\in [0.0, 0.01)$, `max_leaf_nodes` $\in [2, 20)$, `min_impurity_decrease` $\in [0.0, 0.1)$.
- **Region of Competence:** Computed using the k -Nearest Neighbors ($K = 7$).

Imbalance-Aware: Learning-Level Strategies (1/3)

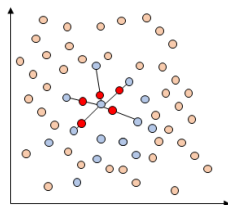
Data-Level Strategy: SMOTE [Araf et al., 2024]

- **Objective:** Synthetically oversamples the minority class within the training set.
- **Mechanism:** Locates the k -nearest neighbors ($k = 5$) for a minority sample and interpolates a new synthetic instance along the connecting line.
- **Outcome:** Achieves a balanced class distribution prior to model training according to a defined oversampling ratio.



Identify k -nearest neighbors
surrounding a minority sample

STEP 1.1



Synthesize new minority samples (marked in red) between
the selected minority sample and its k -nearest neighbors

STEP 1.2

- = Majority sample
- = Minority sample
- = Synthesized sample

Repeat step 1.1 with
other minority
samples until the
desired sample size is
reached

Imbalance-Aware: Learning-Level Strategies (2/3)

Algorithm-Level Strategy: Class-Frequency Weighting [Araf et al., 2024]

- **Objective:** Penalize misclassifications of the minority class by assigning higher weights directly inside the algorithm's loss function.
- **Mechanism:** Weights are automatically derived from the inverse of the class frequencies in the training data.

Implementation Frameworks

- **Scikit-Learn Library (`class_weight="balanced"`):**
 - Computes weights inversely proportional to class frequencies.
 - $\text{class_weight} = \{0: \frac{n_{\text{samples}}}{2 \cdot n_0}, 1: \frac{n_{\text{samples}}}{2 \cdot n_1}\}$
- **XGBoost Library (`scale_pos_weight`):**
 - Used in XGBoost to scale the gradient of the positive class.
 - Calculated as the ratio of negative to positive samples: $\frac{n_0}{n_1}$

Imbalance-Aware: Learning-Level Strategies (3/3)

Algorithm-Level Strategy: Cost-Sensitive Learning [Araf et al., 2024]

- **Objective:** Explicitly guide the loss function during training using predefined, asymmetric clinical costs rather than dataset frequencies.

The Clinical Cost Matrix

	Predicted: ALIVE (0)	Predicted: DEAD (1)
Actual: ALIVE (0)	TN=0	FP=1
Actual: DEAD (1)	FN=10	TP=0

A missed lethal outcome (FN) is penalized **10x** more heavily than a false alarm (FP).

- **Implementation:** `class_weight={0:1, 1:10}` or `scale_pos_weight=10`.

Imbalance-Aware: Decision-Level Strategies

1. Standard Threshold

- Standard ML models use a default threshold of 0.5 to assign classes.
- A 0.5 threshold implicitly assumes that FP and FN have equal clinical cost.

2. Minimum Expected Cost (MEC) [Elkan, 2001]

- MEC computes the optimal threshold based on the misclassification cost.
- MEC selects the class that minimizes the expected misclassification cost.

$$\hat{y} = \arg \min_c \sum_k P(y = k | x) C_{k,c}$$

- $P(y = k | x)$ = Predicted posterior probability.
- $C_{k,c}$ = The specific cost from the matrix (e.g., **FN=10**, FP=1).
- **Result:** Model predictions are strictly aligned with real-world clinical cost.

Imbalance-Aware: Experimental Settings

We systematically compared **7** configurations: **1 standard baseline**, **6 imbalance-aware**.

Systematic Configuration Grid

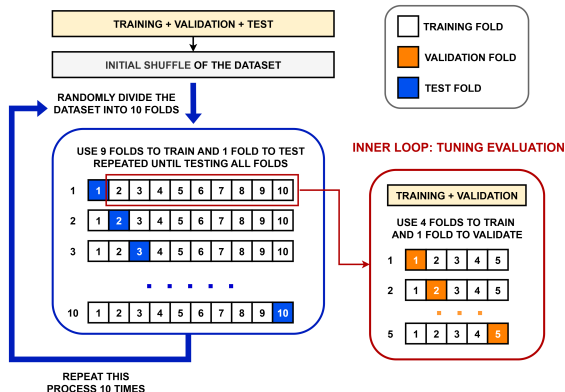
Imbalance Strategy (Learning)	Decision Threshold	Experiment Alias
None (Cost-Insensitive)	Standard (0.5)	→ Baseline
Class-Frequency Weighting	Standard (0.5)	→ Balanced + STD
Class-Frequency Weighting	MEC Policy	→ Balanced + MEC
Cost-Matrix Weighting (1:10)	Standard (0.5)	→ Cost-sensitive + STD
Cost-Matrix Weighting (1:10)	MEC Policy	→ Cost-sensitive + MEC
SMOTE (Data Oversampling)	Standard (0.5)	→ SMOTE + STD
SMOTE (Data Oversampling)	MEC Policy	→ SMOTE + MEC

Learning Pipeline (1/2): Baselines & Static Ensembles

Evaluation Protocol

- **Evaluation (Outer Loop):** Repeated Stratified 10×10 CV.
- **Tuning (Inner Loop):** Stratified 5-Fold CV.
- **Tuning Algorithm:** Random Search ($n_iter=30$).

OUTER LOOP: MODEL EVALUATION



Model-Specific Data Split

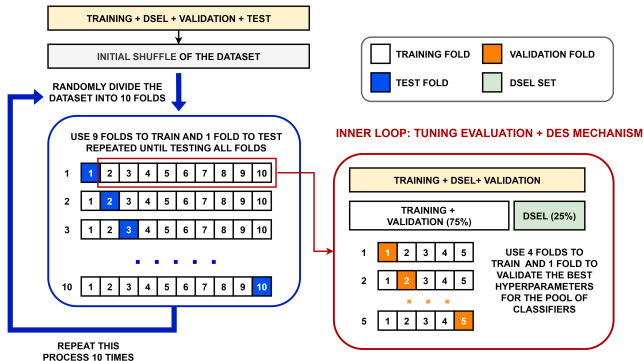
Refitted on the full outer training set and evaluated on the test fold.

Learning Pipeline (2/2): Dynamic Ensembles

OUTER LOOP: MODEL EVALUATION

Evaluation Protocol

- **Evaluation (Outer Loop):** Repeated Stratified 10×10 CV.
- **Tuning (Inner Loop):** Stratified 5-Fold CV.
- **Tuning Algorithm:** Random Search ($n_iter=30$).



Model-Specific Data Split

The outer training set requires a further split: **75% (TRAIN)** to learn and tune the base classifier pool **25% (DSEL)** to fit the competence estimation mechanism.

Evaluation Framework & Metrics

Key Performance Metrics

- **Standard Metrics:** Accuracy, Precision, Recall, F1-score, and ROC-AUC to get a general overview of discriminative power.
- **Primary Metric - Average Cost (AvgCost):** A domain-specific Average Cost per Prediction that directly quantifies asymmetric clinical risk.

$$\text{AvgCost} = \frac{1 \cdot \text{FP} + 10 \cdot \text{FN}}{N}$$

(Lower is better)

Statistical Validation

- We applied the **corrected resampled t-test** on the AvgCost metric to rigorously compare model configurations.
- A p-value < 0.05 was used to determine statistically significant differences.

Conclusions & Future Directions

Key Findings

- **Core Necessity:** Addressing class imbalance via cost-sensitive learning is crucial.
- **Optimal Setup:** Cost-sensitive learning combined with a standard decision threshold yielded the most reliable performance.
- **Top Architectures:** Static ensembles (`StackingClassifier`, `XGBClassifier`) robustly outperformed the best dynamic method (DES-KL).

Limitations & Future Works

- **Data Validation:** Validate the framework on modern, large-scale clinical datasets.
- **Algorithmic Exploration:** Investigate alternative DES pool-generation and threshold calibration techniques to further optimize the FP vs. FN trade-off.

References



Araf, I., Idri, A., and Chairi, I. (2024).

Cost-sensitive learning for imbalanced medical data: a review.

Artificial Intelligence Review, 57(4):80.



Cruz, R. M., Sabourin, R., and Cavalcanti, G. D. (2018).

Dynamic classifier selection: Recent advances and perspectives.

Information Fusion, 41:195–216.



Elkan, C. (2001).

The foundations of cost-sensitive learning.

In *Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 973–978.



Golovenkin, S., Shulman, V., Rossiev, D., Shesternya, P., Nikulina, S., Orlova, Y., and Voino-Yasenetsky, V. (2020).

Myocardial infarction complications [dataset].

UCI Machine Learning Repository.

Thank You for Your Attention

Leonardo Saccotelli

`l.saccotelli@studenti.uniba.it`

Università degli Studi di Bari Aldo Moro